Automazione del Carico Merci Marittimo

Progetto Finale - Ingegneria dei Sistemi Software A.A. 2024/2025

Diego Bruno Marco Crisafulli Sebastiano Giannitti

Università di Bologna - DISI

8 Maggio 2026

Docente: Prof. Antonio Natali

Presentazione del Progetto: TemaFinale25

Obiettivo: Automatizzare le operazioni di carico merci nella stiva di una nave tramite un *Differential Drive Robot* (**cargorobot**).

Entità Principali del Sistema

- **cargoservice**: Orchestratore della logica di business e del processo di carico.
- **productservice**: Servizio esterno per la registrazione dei prodotti (PID e peso).
- **hold**: Area di stoccaggio discretizzata in una griglia con 4 slot disponibili.
- **ioport**: Punto di ingresso merci monitorato da un sensore sonar.
- **sonar:** Gestione delle interruzioni e dei guasti del sensore sonar con segnalazione tramite LED.

Sprint 0: Analisi e Modello Logico

Goal: Formalizzazione dei requisiti e definizione dell'architettura logica di riferimento.

- **Abstraction Gap:** Scelta del metamodello **QAK** per gestire nativamente interazioni asincrone e FSM distribuite.
- **Analisi del Dominio:** Identificazione degli attori core e dei contesti distribuiti (*ctxcargoholdservice*, *ctxcargoservice*, *ctxbasicrobot*, ecc.).
- **Test Plan Preliminare:** Definizione delle condizioni di accettazione per i casi d'uso principali (es. `requesttoload`).

Suddivisione dei Compiti (Sprint 0)

D. Bruno: **Requisiti e Formalizzazione** - Classificazione dei requisiti (CRS, PRD, ROB) e mappatura delle entità del dominio.

M. Crisafulli: **Modellazione QAK** - Progettazione della prima architettura logica e definizione del protocollo dei messaggi.

S. Giannitti: **Problem Analysis & Test Plan** - Analisi delle criticità e stesura dei primi piani di test funzionali.

Sprint 1: Core Business Logic

Goal: Implementazione della logica di business e modellazione dell'ambiente operativo.

- **Clean Architecture (Cat Jump):** Introduzione dell'attore dinamico `productpolice` per isolare il core dal *productservice* esterno.
- **Modellazione Spaziale:** Discretizzazione della stiva in una **Grid Map** 7×6 con coordinate cartesiane per slot, HOME e IOPort.
- **Interazione Cargorobot:** Protocollo di *engagement* dinamico con il *basicrobot24* per l'uso esclusivo durante il carico.

Suddivisione dei Compiti (Sprint 1)

D. Bruno: **Hold Modeling** - Implementazione dell'attore `hold` tramite strutture dati Kotlin e mappatura logica/fisica delle coordinate.

M. Crisafulli: **Clean Architecture** - Sviluppo del componente `productpolice` e gestione del parsing JSON per l'interazione con servizi esterni.

S. Giannitti: **Robot Orchestration** - Definizione della FSM del `cargorobot` e automazione dei test plan per gli edge cases.

Sprint 2: IoT e Integrazione Hardware

Goal: Interfacciamento Raspberry Pi, gestione Sonar/LED e comunicazione MQTT.

- **Reactive Programming:** Modellazione di `sonardevice` come attore streamer che emette stream locali di dati (`sonardata`).
- **Comunicazione MQTT:** Trasmissione asincrona degli eventi sensoriali dal Raspberry al contesto PC tramite broker MQTT.
- **Fault Tolerance:** Logica temporale di 3s per il rilevamento guasti ($D > DFREE$) e attivazione del LED di segnalazione.

Suddivisione dei Compiti (Sprint 2)

D. Bruno: **Hardware Setup** - Configurazione GPIO Raspberry Pi e adattamento script Python di basso livello per Sonar HC-SR04 e LED.

M. Crisafulli: **Network Setup** - Setup del Broker `MQTT` e dell'interfacciamento di comunicazione tra i componenti.

S. Giannitti: **Adapter Logic** - Implementazione del `sonaradapter` per tradurre gli eventi fisici in risposte logiche per l'orchestratore.

Sprint 3: Web-GUI e Monitoraggio

Goal: Visualizzazione dinamica dello stato della stiva e interazione utente.

- **Real-time Update:** Uso di **WebSocket** per spingere i cambiamenti di stato direttamente sulla GUI.
- **User Experience:** Interfaccia intuitiva per il monitoraggio dei prodotti caricati e degli attributi con cui sono registrati.

Suddivisione dei Compiti (Sprint 3)

D. Bruno: **Integration Test** - Verifica della correttezza funzionale dell'integrazione tra più moduli e componenti.

M. Crisafulli: **Produzione dei Deliverable** - Containerizzazione dei componenti, gestione network Docker e file di configurazione.

S. Giannitti: **System Observability e GUI** - Configurazione del sistema a supporto della GUI.

Integrazione AI e Sviluppi Futuri

Goal: Potenziare l'autonomia del *cargorobot* e l'ottimizzazione della stiva tramite tecniche di AI.

- **Ottimizzazione Euristica:** Uso di algoritmi di ricerca per lo *Slot Allocation Problem*.
- **Computer Vision:** Integrazione di modelli di *Object Detection* per identificare i container all'IOPort.
- **Natural Language Interface:** Interazione con la Web-GUI tramite LLM.